# QR Code Menu System

## Detailed Project Report

*A smart digital ordering solution for restaurants & coffee shops*

| | |
|---|---|
| **Document Type** | Project Planning & Technical Report |
| **Date** | March 2026 |
| **Version** | 2.0 — Updated with Security & Enhanced Features |

## Table of Contents

## 1. Executive Summary

This report outlines the full design, architecture, and development plan for a QR Code-based digital menu and order management system for restaurants and coffee shops. The system eliminates the inefficiencies of traditional paper menus and manual order-taking by allowing customers to browse a rich digital menu and place orders directly from their mobile device with no app download required.

Customers scan a unique QR code on their table, browse a visual menu with food photos, add items to their cart, write notes, and submit their order. The order instantly appears on a real-time dashboard where staff manage all tables, track order statuses, and respond to customer needs.

The system is built on a modern, cost-effective technology stack — Next.js, Supabase, Vercel, GitHub, and optionally Google AI Studio — and includes robust security mechanisms to prevent order fraud and unauthorized access.

# 2. Problem Statement

## 2.1 The Core Problem

In traditional restaurants and cafes, order-taking is one of the biggest bottlenecks in service. Customers wait for a waiter, verbally communicate their order, and the waiter manually records or enters it — a process prone to delays, miscommunication, and errors.

- **Waiting time:** Customers wait several minutes before being acknowledged by a waiter.
- **Miscommunication:** Orders are misheard or misrecorded, leading to wrong dishes.
- **Rush hour chaos:** During peak hours, staff are overwhelmed and service quality drops.
- **No menu photos:** Paper menus lack visuals, limiting the customer's ability to choose confidently.
- **No order visibility:** Customers have no way to track their order status after placing it.

## 2.2 Security Problems (Newly Identified)

⚠️ **Problem 1 — Bad Client Runs Away Without Paying**

A customer places an order using the QR code, the kitchen prepares the food, and the customer leaves before paying. Since there is no mandatory payment step, this is a real risk for any contactless ordering system.

⚠️ **Problem 2 — Orders Placed From Outside the Restaurant**

Because QR codes contain a static URL, anyone who scans the code once can share the link or bookmark it and place orders from home or anywhere else — causing the restaurant to prepare food for nobody and lose money.

# 3. Solution Overview

The QR Menu System is a web-based platform with two distinct interfaces: a customer-facing menu and a staff-facing management dashboard. Both are connected through a real-time database that ensures orders and updates are reflected instantly across all devices.

## 3.1 Customer Journey

| Step | Action | Detail |
|---|---|---|
| 1 | **Scan QR Code** | Staff must first activate the table. Customer scans QR and sees the live menu. |
| 2 | **Browse Menu** | Mobile-friendly menu with categories, food photos, |

| | | |
|---|---|---|
| | | descriptions, and prices. |
| 3 | **Add to Cart** | Select items, quantities, and add optional notes per item (e.g., no onions). |
| 4 | **Place Order** | Customer submits order — it appears instantly on the kitchen dashboard. |
| 5 | **Track Status** | Customer sees live status: Received, Preparing, or Ready. |
| 6 | **Pay & Leave** | Customer pays by cash or card to the waiter. Staff closes the table session. |

### 3.2 Staff Dashboard Features

- **Live Order Feed:** New orders appear instantly with a visual alert.
- **Table Overview:** Grid view of all tables showing active sessions and order status.
- **Order Details:** Table number, items, quantities, customer notes, and elapsed time.
- **Order Status Control:** Move orders through: Received → Preparing → Ready → Delivered.
- **Call Waiter Alerts:** Real-time notification when a customer requests assistance.
- **Session Management:** Open and close table sessions — the core security control.
- **Menu Management:** Add, edit, or deactivate menu items without any technical knowledge.
- **Order History:** Full log of all completed orders for end-of-day review.

## 4. Security System — Table Session Management

🔒 **Core Security Principle**

The QR code on a table is never directly active. It only works when a staff member has opened a session for that table. Without an active session, scanning the QR code shows a message asking the customer to call a waiter. This one mechanism solves both the fraud and remote-ordering problems.

### 4.1 How Table Sessions Work

A table session is a time-limited authorization token that a staff member creates when a real customer sits down. The entire ordering flow depends on this session being active.

#### Session Lifecycle

| Stage | Who Acts | What Happens |
|---|---|---|

| | | |
|---|---|---|
| **Customer arrives** | Staff member | Staff taps 'Open Table' on dashboard — a unique session token is created in the database for that table. |
| **Customer scans QR** | System | QR URL checks for an active session. If found, the menu loads. If not, customer sees 'Please ask your waiter'. |
| **Customer orders** | System | Orders are accepted only while session is active. Each order is linked to the session ID. |
| **Customer pays** | Staff member | Staff taps 'Close Table' — session ends, QR code stops working, table marked as empty. |
| **Auto-expiry** | System | If staff forgets to close, the session auto-expires after 3 hours as a safety net. |

## 4.2 Database — table_sessions Table

A new table is added to the database specifically for session management:

| Column | Purpose |
|---|---|
| `id` | Unique session ID — included in every order for traceability |
| `table_id` | Which table this session belongs to |
| `opened_by` | Staff member ID who opened the session |
| `opened_at` | Timestamp of when session was opened |
| `closed_at` | Null while active, set when staff closes it |
| `expires_at` | Auto-set to opened_at + 3 hours as a safety expiry |
| `status` | active / closed / expired |

## 4.3 Additional Security Layers

- **Order approval mode (optional):** Staff can enable a setting where the first order from a table requires a waiter to approve it before going to the kitchen — an extra human verification step.
- **Order limit per session:** The system can cap the maximum number of orders per session (e.g., 10 rounds) to prevent abuse.
- **Session audit log:** Every session open/close is logged with staff ID and timestamp, creating full accountability.
- **IP + User Agent logging:** Each order records the device fingerprint — useful for investigating suspicious patterns.

### 4.4 Security Problem Resolution Summary

| Risk | Solution |
|---|---|
| Customer orders and runs away | Session closed immediately by staff. Optional first-order approval adds a human check before kitchen starts preparing. |
| Someone orders from home via saved URL | No active session exists remotely. QR link without a staff-opened session returns a 'Table not active' screen. |
| Staff forgets to close session | Auto-expiry closes all sessions after 3 hours automatically. |
| Waiter impersonation or unauthorized access | All session actions require staff authentication via Supabase Auth — logged in with password. |

## 5. Enhanced Features

Beyond the core ordering and dashboard functionality, the system includes a set of value-added features that significantly improve the experience for both customers and restaurant staff.

### 5.1 Call Waiter Button

A persistent button on the customer menu page allows customers to request assistance from a waiter without waving or shouting. When pressed, an instant notification appears on the staff dashboard showing the table number and the time of the request. This is useful for customers who need help with the menu, have an accessibility need, or simply want to ask a question. The waiter can mark the request as 'handled' from the dashboard to dismiss the alert.

### 5.2 Split Bill Feature

At checkout, customers can request to split the bill between a defined number of people. The system automatically calculates the equal share per person and displays each individual amount clearly. This is one of the most common pain points in group dining and cafes — solving it digitally removes awkward manual calculations and speeds up table turnover.

### 5.3 Loyalty & Points System

Customers can optionally create a simple account linked to their phone number. With each order, they accumulate loyalty points based on their total spend. Points can be redeemed for discounts on future visits. For restaurants, this creates a direct incentive for repeat visits and gives the owner valuable data about their most loyal customers. The loyalty account is lightweight — no password needed, just a phone number and OTP verification via Supabase Auth.

## 5.4 Estimated Wait Time

The kitchen dashboard tracks how long each order takes from placement to completion. Over time, the system builds an average preparation time per order. This average is then used to display a dynamic estimated wait time to the customer after they place their order. The estimate automatically adjusts based on how many orders are currently being prepared — if the kitchen is busy, the estimate increases accordingly. This simple feature dramatically reduces customer anxiety and the number of 'is my food ready?' inquiries.

## 5.5 Analytics Dashboard

The analytics section gives the restaurant owner actionable business intelligence derived from order data. All of this is generated automatically with no extra effort from the owner.

- **Top-selling items:** Ranked list of most ordered dishes by day, week, or month.
- **Busiest hours:** Bar chart showing order volume by hour of day — useful for staffing decisions.
- **Average order value:** Track how much customers typically spend per visit over time.
- **Daily and weekly revenue:** Simple revenue tracking without needing a separate accounting tool.
- **Table utilization:** Which tables are most active, how long sessions last on average.
- **Most-used notes:** What customization requests customers make most often (e.g., 'extra sauce').

## 5.6 Low Stock Alerts

When adding a menu item, the owner can optionally set a 'daily quantity limit' — for example, only 20 portions of a special dish are available each day. As orders come in, the system counts down the remaining quantity. When the count reaches a low threshold, the owner and kitchen staff receive a warning notification on the dashboard. When quantity hits zero, the item is automatically marked as unavailable on the customer menu. This prevents overselling limited items and avoids disappointing customers.

## 5.7 Allergen and Dietary Filters

Each menu item can be tagged with dietary labels such as Vegetarian, Vegan, Gluten-Free, Dairy-Free, Spicy, Contains Nuts, and Halal. Customers can filter the menu by these tags to quickly find items that meet their dietary requirements or restrictions. This is increasingly important as awareness of food allergies and dietary preferences grows worldwide. It also reduces the need for customers to ask staff about ingredients, speeding up the ordering process.

### 5.8 Favorites and Reorder

Returning customers with a loyalty account can see a 'Your Favorites' section on the menu showing items they have ordered before, ranked by frequency. They can add an entire previous order to their cart with a single tap. This feature significantly speeds up the ordering process for regular customers and reinforces loyalty by making the app feel personalized to each individual.

### 5.9 AI-Powered Upsell Suggestions

Using the Google Gemini API, the system can intelligently suggest complementary items to customers as they build their cart. For example, a customer who adds a burger might see 'Customers who ordered this also added: Fries, Coleslaw, or a Milkshake.' These suggestions are based on real order history from the database and powered by AI to find meaningful patterns. This feature increases the average order value without any effort from the staff.

### 5.10 Multi-Branch Support

Restaurant owners who operate more than one location can manage all branches from a single owner account. Each branch has its own menu, tables, QR codes, and staff accounts. The analytics dashboard can show combined or per-branch views. This makes the system scalable for small chains or franchise owners who want a unified management platform.

## 6. Technology Stack

The system is built on four modern, well-supported tools that work seamlessly together. All four have generous free tiers making the initial build cost-free.

#### Next.js — Frontend Framework

- React-based framework ideal for both customer menu and staff dashboard
- Server-side rendering for fast initial page loads on customer devices
- Built-in API routes — simple backend logic lives in the same codebase
- Works natively with Vercel — zero-configuration deployment
- Large ecosystem of UI libraries and strong community support

#### Supabase — Backend, Database, Auth & Storage

- PostgreSQL database for structured management of menus, orders, tables, and sessions
- Real-time subscriptions — dashboard updates instantly when a new order is placed

- Built-in authentication for staff login and customer loyalty accounts
- Supabase Storage — upload and serve food images with auto-generated public URLs
- Row Level Security — restaurant owners only see their own data
- Auto-generated REST API — no need to build a backend from scratch

### Vercel — Hosting & Deployment

- One-click deployment — every GitHub push deploys automatically
- Global CDN — fast loading for customers anywhere in the world
- Free SSL certificate — all traffic is encrypted
- Preview deployments — every pull request gets its own URL for testing
- Built-in analytics to monitor performance and visitor activity

### GitHub — Version Control & Collaboration

- Store all source code securely with full history of every change
- Branching — work on new features without affecting the live production app
- Directly connected to Vercel for automatic deployment on every commit
- Foundation for future CI/CD pipelines and automated testing

### Google AI Studio (Gemini API) — AI-Powered Intelligence Layer

- Auto-generate food item descriptions from uploaded photos
- Multi-language menu translation for international guests
- Smart upsell suggestions based on real order history
- Image moderation — verify uploaded images actually contain food
- Natural language search — customers describe what they want, AI finds it
- Smart dietary tagging — suggests tags based on item name and image

## 7. Database Design

The database is built in PostgreSQL via Supabase. The schema now includes six core tables to support the full feature set including session management, loyalty, and enhanced orders.

### restaurants

Restaurant profile and configuration

| Column | Description |
|---|---|
| id | |
| name | |
| logo_url | |
| address | |
| owner_id | |
| created_at | |

## tables

Physical tables in the restaurant

| Column | Description |
|---|---|
| id | |
| restaurant_id | |
| table_number | |
| qr_code_url | |
| status | (empty/occupied/ordering) |

## table_sessions

Active session per table — core security mechanism

| Column | Description |
|---|---|
| id | |
| table_id | |
| opened_by | (staff_id) |
| opened_at | |
| closed_at | |
| expires_at | |
| status | (active/closed/expired) |

## menu_items

All food and drink items on the menu

| Column | Description |
|---|---|

| | |
|---|---|
| id | |
| restaurant_id | |
| category | |
| name | |
| description | |
| price | |
| image_url | |
| tags | (json) |
| daily_limit | |
| is_available | |
| created_at | |

## orders

Each order placed by a customer at a table

| Column | Description |
|---|---|
| id | |
| table_id | |
| session_id | |
| restaurant_id | |
| status | (received/preparing/ready/delivered) |
| total_price | |
| notes | |
| created_at | |

## order_items

Individual line items within each order

| Column | Description |
|---|---|
| id | |
| order_id | |
| menu_item_id | |
| quantity | |

| | |
|---|---|
| `item_notes` | |
| `unit_price` | |

### loyalty_accounts

Optional customer loyalty profiles

| Column | Description |
|---|---|
| `id` | |
| `phone_number` | |
| `total_points` | |
| `created_at` | |
| `last_visit` | |

# 8. Food Image Upload System

Menus with food photography consistently generate higher order rates than text-only menus. Customers make decisions visually, and showing a high-quality photo of a dish significantly increases its likelihood of being ordered.

## 8.1 Upload Flow

1. Owner opens the 'Add Item' form in the menu management dashboard.
2. Owner fills in item name, category, description, price, tags, and daily limit.
3. Owner selects a photo from their phone or computer (jpg, png, webp — max 5MB).
4. System validates the file type and size before uploading.
5. File uploads to Supabase Storage under the food-images/ bucket.
6. Supabase returns a permanent public URL for the image.
7. The URL is saved in the menu_items table alongside all other item data.
8. The customer menu fetches the URL and displays the image with lazy loading.

## 8.2 AI Enhancement (Google Gemini)

- **Auto-description:** Owner uploads photo — Gemini analyzes the image and writes a compelling food description automatically.
- **Smart tagging:** AI suggests dietary tags (Vegetarian, Gluten-Free, Spicy, etc.) based on the image and name.

- **Image moderation:** AI verifies the uploaded image actually contains food before saving to storage.

# 9. QR Code System

Each table has a unique QR code that links to that table's menu page. Combined with the session system, the QR code is only active when a staff member has opened a session — preventing remote or unauthorized ordering.

## 9.1 QR Code URL Format

```
https://yourapp.com/menu?table=TABLE_UUID&restaurant=RESTAURANT_UUID
```

When a customer scans this URL, the system first checks for an active session on that table. If no session exists, the customer sees a friendly 'Please ask your waiter to activate your table' message. If a session is active, the full menu loads immediately.

## 9.2 QR Code Management

- QR codes are generated automatically when a new table is added
- Codes can be downloaded as PNG images for printing on cards, stickers, or table stands
- If a QR code is lost or damaged, a new one can be regenerated instantly — the table UUID stays the same

# 10. Real-Time Order System

When a customer places an order, the staff dashboard updates immediately — no manual refresh needed. This is powered by Supabase Realtime, which broadcasts database changes to all connected clients via WebSockets.

## 10.1 Order Status Flow

| | | | |
|---|---|---|---|
| 📥 **Received** | 👨‍🍳 **Preparing** | ✅ **Ready** | 🍽️ **Delivered** |

# 11. Project Folder Structure

```
  /qr-menu-system   ├── /app   │   ├── /menu              ← Customer menu (QR link
target — session-gated)   │   ├── /dashboard         ← Staff dashboard (auth
protected)   │   │   ├── /orders        ← Live order feed   │   │   ├── /tables
← Table + session management   │   │   ├── /menu-items    ← Add/edit/remove food items
│   │   ├── /analytics     ← Revenue and order insights   │   │   └── /settings      ←
Restaurant profile & branch config   │   └── /api               ← API routes (auth,
orders, sessions)   ├── /components             ← Shared UI components   ├── /lib   │
├── supabase.js        ← Supabase client + realtime   │   ├── sessions.js        ←
Session open/close/validate logic   │   └── gemini.js          ← Google AI helpers
└── /public                ← Static assets
```

# 12. Development Roadmap

The project is broken into three phases. Each phase produces a working, testable product that can be tested with real restaurants.

## Phase 1 — Core MVP (Weeks 1–4)

- Set up GitHub, Supabase project, and Vercel deployment pipeline
- Build database schema including the table_sessions security table
- Implement staff authentication via Supabase Auth
- Build table session open/close flow on dashboard
- Build QR code generation per table with session validation gate
- Build customer-facing menu page (browse, add to cart, place order)
- Build basic staff dashboard — live orders, order status updates

## Phase 2 — Enhanced Features (Weeks 5–8)

- Add food image upload via Supabase Storage
- Add Call Waiter button and dashboard alert system
- Add order status tracking for customers
- Add allergen and dietary filter tags
- Add low stock alerts and daily quantity limits
- Add session audit log for accountability
- Add split bill calculation feature
- Mobile UI polish and performance optimization

## Phase 3 — AI, Analytics & Loyalty (Weeks 9–12)

- Integrate Google Gemini for auto-description and smart tagging

- Build analytics dashboard (top items, busiest hours, revenue)
- Build loyalty account system (phone number + OTP)
- Add favorites and reorder feature for returning customers
- Add AI upsell suggestions in cart
- Add estimated wait time display
- Add multi-branch support for owners with multiple locations

## 13. Cost Estimate

All four tools in the stack have free tiers sufficient to run a production app at small to medium scale.

| Service | Free Tier | Paid Starts At | Notes |
|---|---|---|---|
| Supabase | 500MB DB, 1GB storage | $25/month | More than enough for MVP |
| Vercel | 100GB bandwidth | $20/month | Free tier handles most small restaurants |
| GitHub | Unlimited repos | Free | No cost needed |
| Google AI Studio | 15 req/min free | Pay per use | Very cheap for AI features |

## 14. Conclusion

The QR Code Menu System is a technically sound, commercially valuable, and practically complete product concept. It solves a real, everyday problem faced by thousands of restaurants and cafes — slow and error-prone ordering — with a modern, elegant, and cost-effective solution.

What sets this system apart from simpler alternatives is its focus on security and operational trust. The table session mechanism ensures that the restaurant is always in control — no unauthorized orders, no remote abuse, full accountability. This is a critical design decision that protects the business model in markets where digital payment is not yet widely available.

The enhanced features — from the call waiter button and split bill tool to the loyalty system and AI upsells — transform this from a simple ordering app into a comprehensive restaurant operations platform that adds value at every touchpoint.

With a clear three-phase development roadmap, a zero-cost starting stack, and a well-defined architecture, this product is ready to be built. The foundation is solid, the problems are real, and the solution is practical.